

CNN for MNIST Digit Classification

CYSE 635 — Module 02 Project Report

Student Name: Bongiwe Mkwanaenzi

Date Submitted: 10/02/2026

Course / Section: CYSE 635

1. Introduction

The project objective was to design and train a CNN to classify handwritten digits from MNIST, then analyze the network's behavior through evaluation and visualizations. The project prompt required at least two convolutional layers and a fully connected layer, so this report is based on a network with three convolutional layers and a fully connected layer that uses dropout as a regularization method. The model framework used is PyTorch, and the network was trained on the 60,000 image training split for ten epochs with Adam, then evaluated on the 10,000 image test split. It reached 99.17 percent test accuracy with 104,202 trainable parameters.

2. Model Implementation

2.1 Architecture Overview

The network was implemented in PyTorch with three convolutional layers and one fully connected layer. Each convolutional layer applies a 3 by 3 filter with stride 1 and padding 1, followed by a ReLU activation and a 2 by 2 max pooling layer, and the number of filters widens from 32 to 64 to 128 across the three layers. The classifier flattens the result, applies dropout and passes it to a single fully connected layer with 10 outputs, one for each digit class.

A 3 by 3 filter is the smallest kernel with a center pixel and a full ring of neighbors, so it is the smallest size that can encode an oriented intensity change, and it does so with nine weights per input channel. One such filter responds only to a local intensity transition, and stroke-level features appear further up the network as each layer widens the receptive field, so stacking three small filters reaches a wide receptive field with fewer weights than one large filter and adds a non-linearity between each stage. Padding of 1 keeps each output at its input size, so pixels at the border are convolved as often as pixels at the center. ReLU was chosen because the network needs a non-linearity to represent patterns that are not a weighted sum of pixels, and unlike sigmoid or tanh its gradient does not fall toward zero for large inputs, which keeps the gradient signal from weakening across three layers. Max pooling at 2 by 2 keeps the largest activation in each window and discards the other three, which halves each spatial dimension and makes the response tolerant to shifts of roughly one pixel per pooling stage.

The filter count doubles at each stage because the feature maps shrink, so the representation gains depth as it loses spatial detail. A third convolutional layer was used because the extra pooling stage reduces the flattened vector from 3,136 values to 1,152 and therefore shifts most of the model's parameters out of the fully connected layer and into convolution, where weight sharing constrains them. One fully connected layer was then enough, because the convolutional layers already produce features that separate the classes, and the output layer has 10 units with no softmax applied, since the cross entropy loss function applies it internally during training.

2.2 Regularization Techniques

Dropout is the only regularization method used in this network. A dropout layer with a rate of 0.5 sits after the flatten operation and before the fully connected layer, so half of the 1,152 incoming activations are set to zero at random on each training step. The fully connected layer is the only layer that sees the whole representation at once, which makes it the layer able to memorize feature combinations belonging to individual training images, while the convolutional layers are already constrained by weight sharing because each filter must be useful at every position in every image.

The rate of 0.5 is a common default for a dense layer and was used without tuning, since no comparison across rates was run in this project. At that rate the classifier trains on a different random half of its 1,152 inputs at every step, so it cannot settle on any single feature as decisive. Dropout is active only while the model is in training mode, and PyTorch disables the dropping and rescales the remaining activations when the model is switched to evaluation mode, which is why the evaluation cells call that method before measuring.

3. Data & Preprocessing

The MNIST dataset was loaded through the torchvision dataset library, which provides the standard split of 60,000 training images and 10,000 test images. Each image is a 28 by 28 grayscale picture of a single handwritten digit with a label from 0 to 9. The training set was downloaded into a local data directory on the first run and the test set was read from that same directory.

Preprocessing consists of a single step, a tensor conversion applied to every image as it is loaded. It converts each image into a tensor of shape 1 by 28 by 28 and rescales the pixel values from the 0 to 255 integer range into the 0 to 1 floating point range. No mean and standard deviation normalization was applied, and no augmentation was used, so each image reaches the network in its original orientation on every epoch.

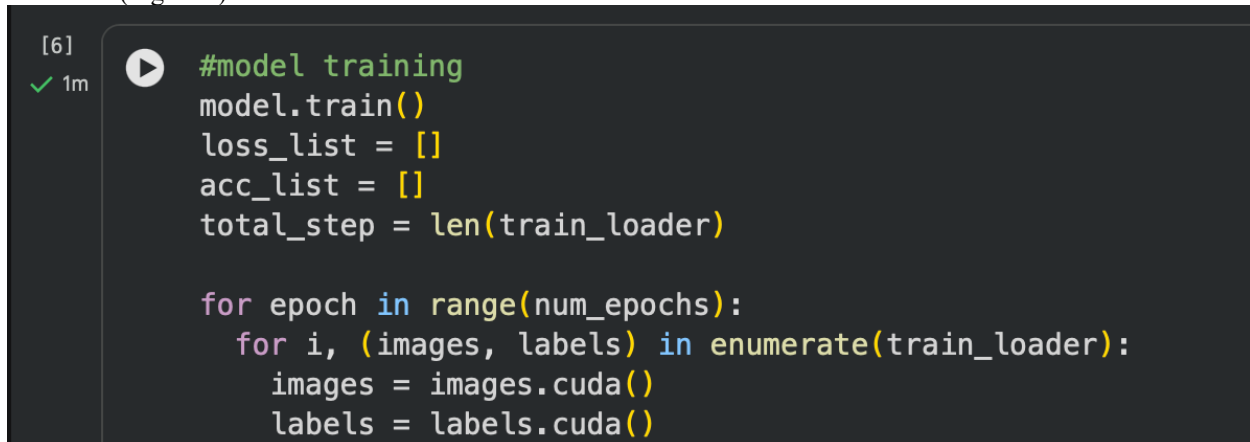
Both splits were wrapped in data loaders with a batch size of 256 and four worker processes for loading. The training loader shuffles the data at the start of each epoch and the test loader does not. At 256 images per batch the training set yields 235 batches per epoch, the last of which holds 96 images.

4. Training

The model was trained with the Adam optimizer at a learning rate of 0.001 for ten epochs, using cross entropy loss as the objective and the batch size of 256 set in the data loaders. Adam was chosen because it adapts a step size for each parameter from that parameter's own gradient history, so the learning rate needs far less tuning than plain stochastic gradient descent would require on the same problem. Cross entropy loss is the standard objective for single label multiclass classification, and it combines the softmax and negative log likelihood steps into one operation, which is why the network outputs raw scores rather than probabilities.

Each training step moves a batch of images and labels to the GPU, passes the images through the network, computes the loss against the labels, clears the gradients left from the previous step, backpropagates and updates the weights. Clearing the gradients is required because PyTorch adds each new gradient to whatever is already stored rather than replacing it, so without that step every update would be computed from the running sum of all earlier batches. Training ran on a GPU runtime and took approximately one minute for all ten epochs, as seen below in the

screenshot(Figure 1).



```
[6] ✓ 1m #model training
model.train()
loss_list = []
acc_list = []
total_step = len(train_loader)

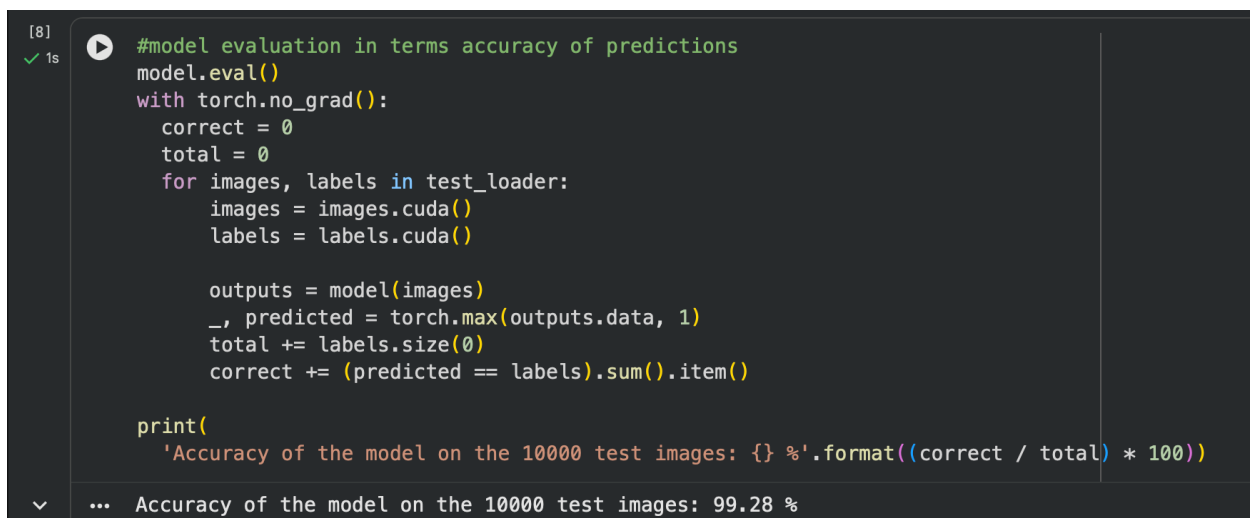
for epoch in range(num_epochs):
    for i, (images, labels) in enumerate(train_loader):
        images = images.cuda()
        labels = labels.cuda()
```

Figure 1, showing snippet with training runtime in minutes

5. Testing & Evaluation

5.1 Performance Metrics

The trained model was evaluated on all 10,000 test images with the network in evaluation mode, so dropout was inactive and the full representation was available. Each image was assigned the class with the highest output score, and that prediction was compared against the image's label. The model classified 99.28%(see figure 2 below) of the test set correctly, which is 9,928 of the 10,000 images and leaves 72 misclassified.



```
[8] ✓ 1s #model evaluation in terms accuracy of predictions
model.eval()
with torch.no_grad():
    correct = 0
    total = 0
    for images, labels in test_loader:
        images = images.cuda()
        labels = labels.cuda()

        outputs = model(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    print(
        'Accuracy of the model on the 10000 test images: {} %'.format((correct / total) * 100))

... Accuracy of the model on the 10000 test images: 99.28 %
```

Figure 2. Showing the accuracy % of the model on the test images

5.2 Per-Class Results

Every image the model misclassified was displayed with its true label and the label the model assigned. The evaluation pass separates each batch of test predictions into those that match their labels and those that do not,

collects both groups across all 40 test batches, and renders them as grids of twenty images. At 72 errors the grid fits the full set, so nothing is sampled(see Figure 3 below).

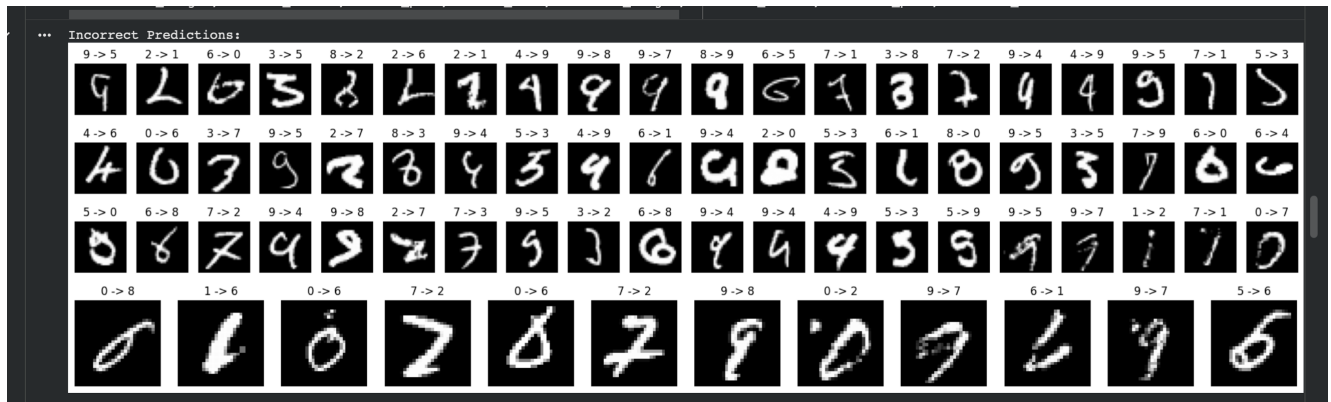


Figure 3, showing misclassified data

The errors are not spread evenly across the ten digits. The digit 9 accounts for 19 of the 72, more than a quarter of all errors, and is most often read as a 4 or a 5. The next most missed digits are 7 and 6 with 9 errors each, then 5 with 7. At the other end the model misses a 1 only twice and an 8 four times. The labels the model assigns in place of the true one are spread far more evenly, from 5 for the digit 0 up to 9 for the digit 5, so the errors concentrate in which digits get missed rather than in which labels get over-assigned. The most frequent individual substitutions are a 9 read as a 4 and a 9 read as a 5, at six occurrences each. Counting the pair in both directions, a 4 read as a 9 together with a 9 read as a 4 accounts for 10 of the 72 errors.

6. Performance Analysis

6.1 Overfitting / Underfitting

Loss and accuracy were recorded at every training step and plotted across the full run, which gives one point per batch rather than one per epoch, so each curve spans 2,350 points.

Batch accuracy begins at about 10 percent and the loss at about 2.3, which is what uniform guessing across ten classes produces. Accuracy passes 90 percent within the first fifty steps and settles into a band near 98 to 99 percent from around step 500, while the loss drops below 0.1 over the same stretch and stays there. Both curves flatten rather than turning back upward, and the width of the remaining band reflects the measurement rather than the model, since each point describes one batch of 256 images with dropout active.

A training curve alone cannot show overfitting, which is a divergence between performance on trained and untrained data. The comparison that settles it is the 99.28 percent test accuracy against the late training band of 98 to 99 percent. The test figure sits at or slightly above that band because training accuracy is measured with dropout active, so there is no gap in the direction overfitting would produce, and accuracy is too high on both splits for underfitting. In conclusion, the model is a good fit.

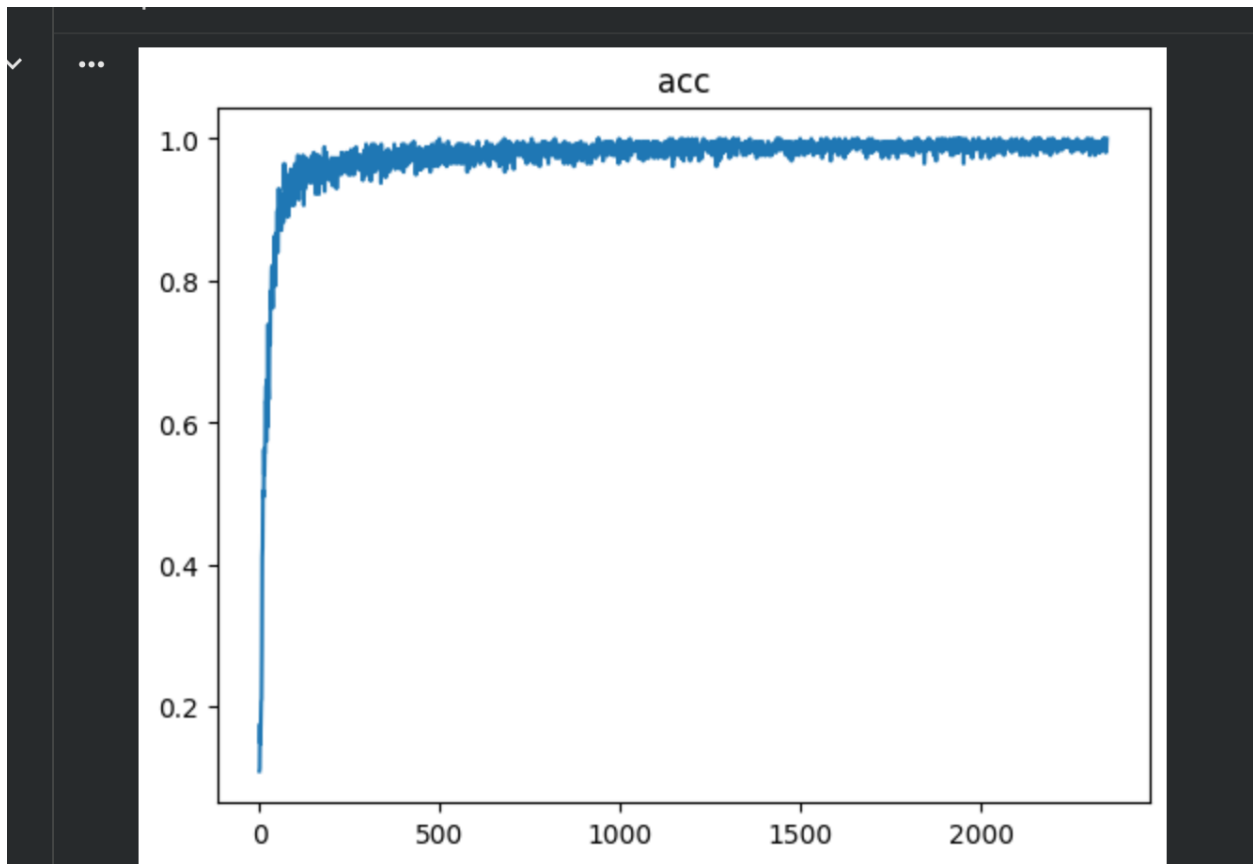
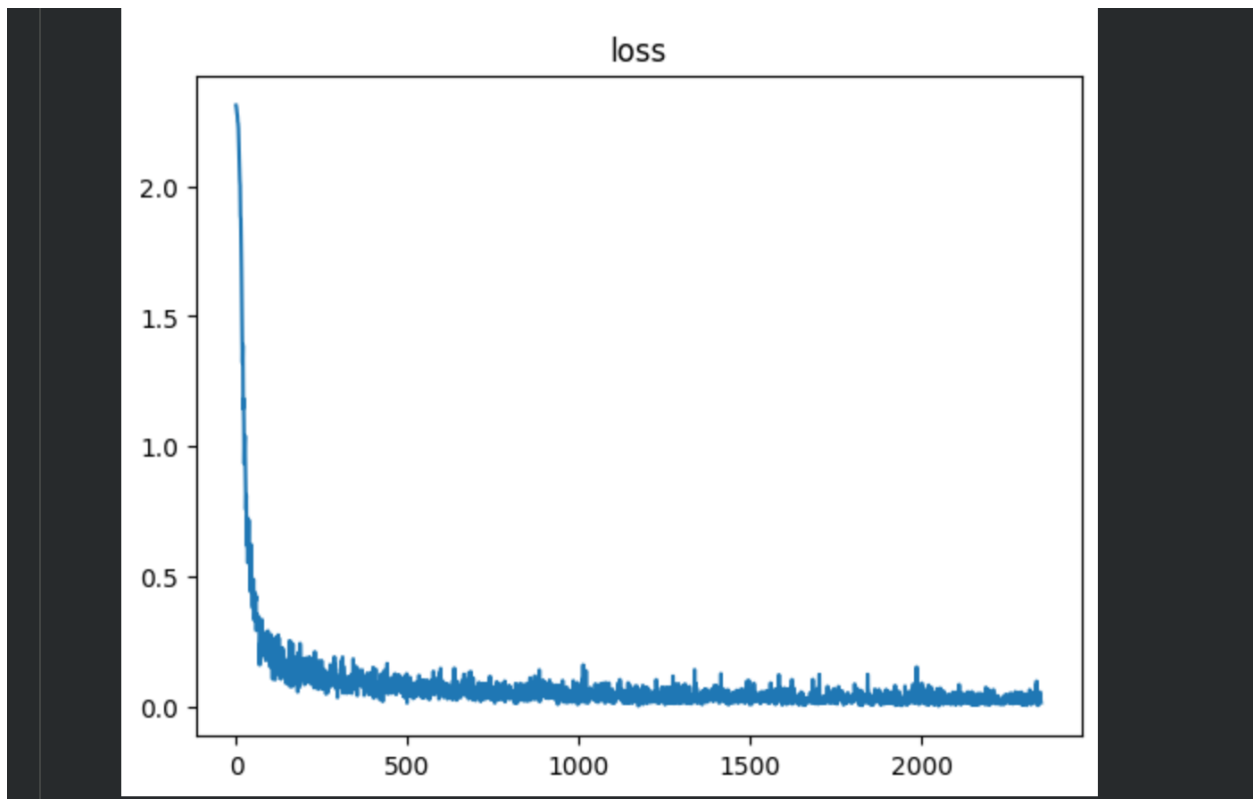


Figure 4 and 5, *Batch accuracy (above) and batch loss (below) across training steps.*



6.2 Filter Visualization.

The learned filters were read from the weight tensors of the three convolutional layers. The first layer holds 32 filters of shape 1 by 3 by 3, so each is displayed as stored, while the second and third layers hold 32 and 64 input channels per filter and were averaged across those channels to be shown as single images. That averaging removes the channel specific structure that gives a deeper filter its selectivity, so the deeper grids are harder to read than the first.

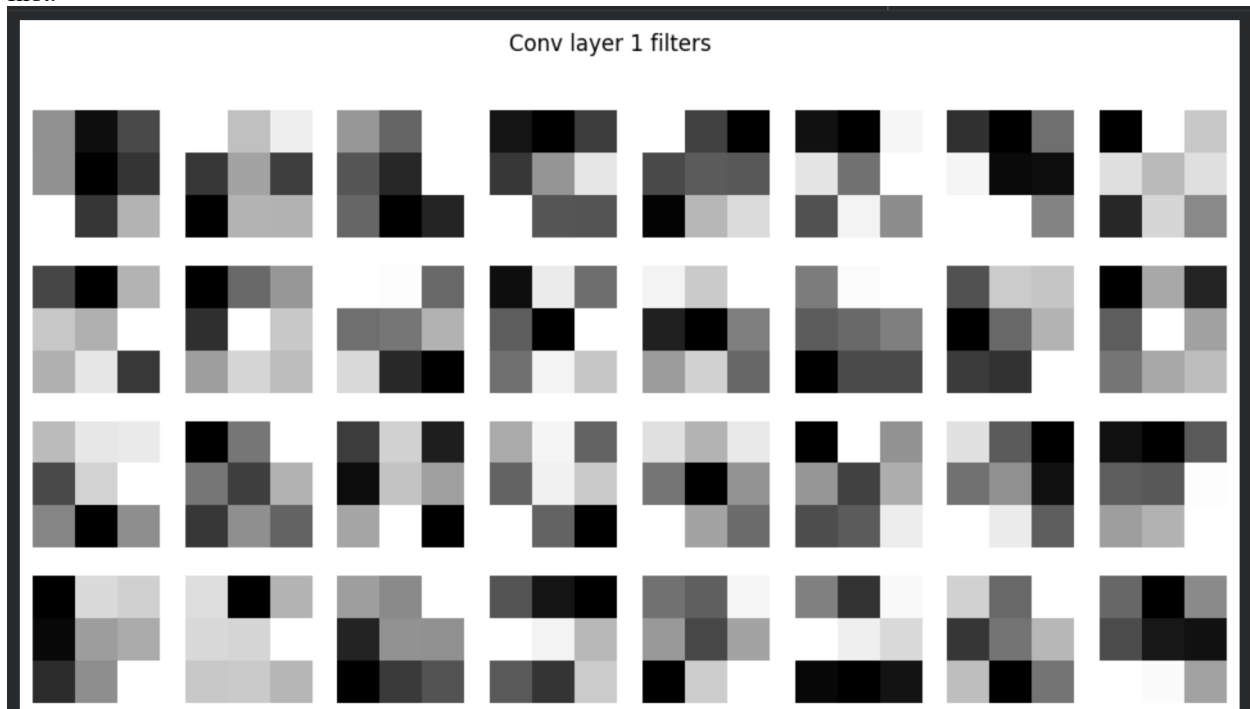


Figure 6, *Conv layer 1*

The first layer filters show strong contrast across all 32. Most are oriented intensity transitions, with a dark band on one side of the kernel and a light band on the other, covering vertical, horizontal and both diagonal directions. A few are blob patterns with a bright middle against a darker ring or the reverse, which respond to isolated bright points rather than to direction. None are flat.

The second and third layer grids are visibly lower in contrast with mid gray dominating and far less clear structure, which follows both from the channel averaging and from deeper filters responding to combinations of earlier features rather than to raw pixel arrangements.

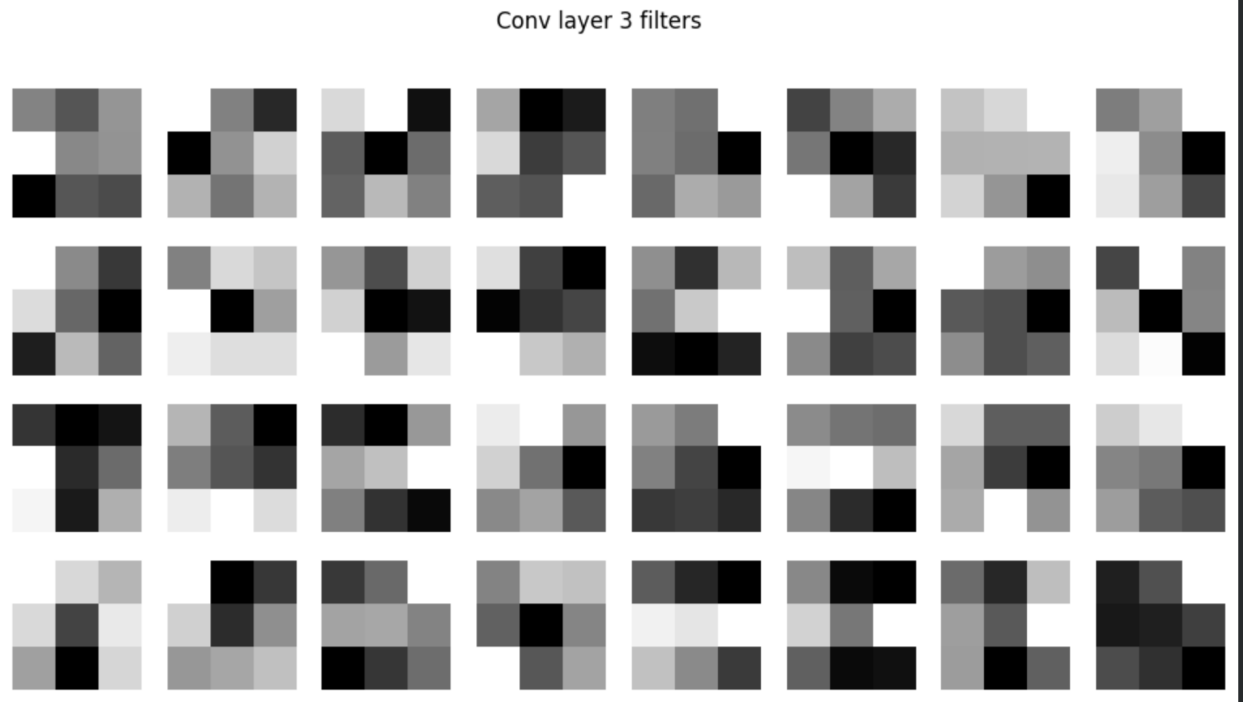
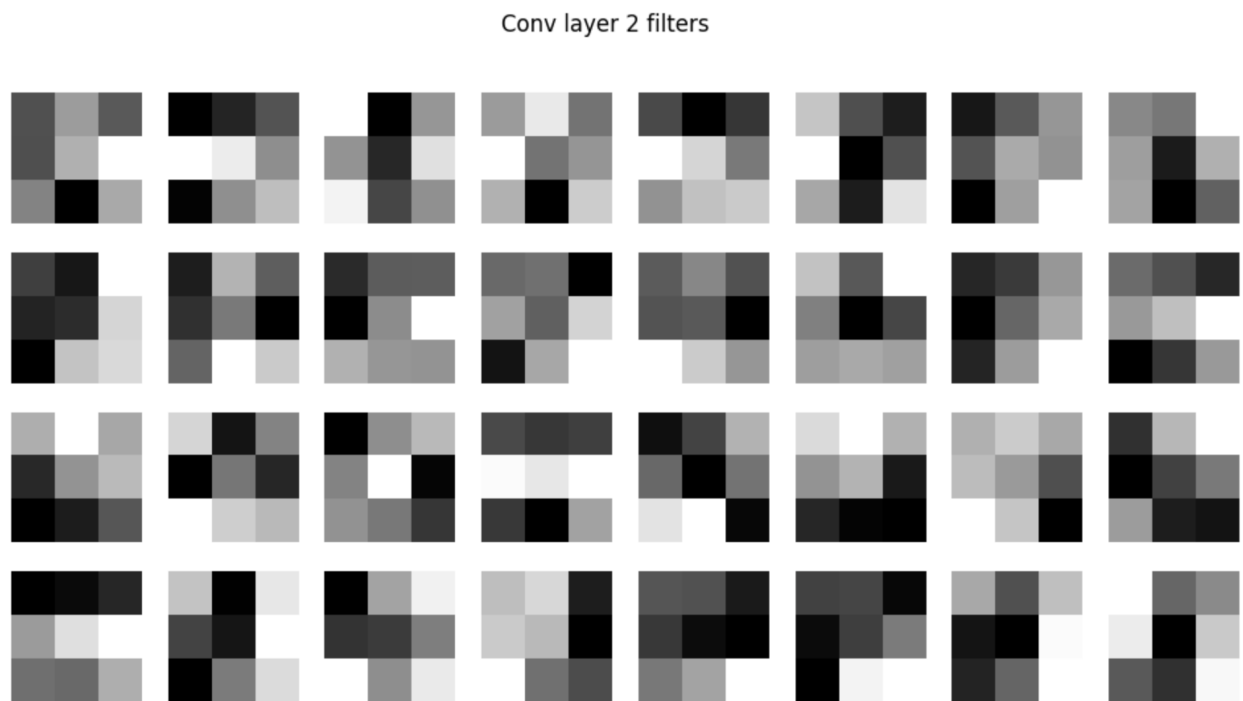


Figure 7 and 8, *Convolutional layers 2(above) and 3(below)*



The feature maps were produced by passing one test image, a handwritten 7, through progressively longer slices of the convolutional stack, each cut after an activation and before the pooling that follows it. This gives 32 maps at 28 by 28 for the first layer, 64 at 14 by 14 for the second and 128 at 7 by 7 for the third, with the first 32 of each displayed.

NB: Feature maps visuals have been added to appendix and can also be viewed in the notebook shared.

At the first layer every map is still recognizably a 7, with individual channels brightening different strokes and some rendering the digit dark against a lighter background. At the second layer the maps have specialized and the digit has separated into its parts, with some channels responding almost only to the horizontal top bar and others almost only to the diagonal stroke. At the third layer the maps are sparse, mostly dark with a few bright cells marking where a feature was found, and the 7 is no longer visible as a shape.

Several third layer maps are entirely dark for this input. That indicates those filters do not respond to a 7 rather than that they are dead, since a dead filter would stay inactive for every input and these were tested against one image only.

6.3 Confusion Matrix Analysis

The errors concentrate in digit pairs that share stroke geometry, which explains why 9 is missed far more often than any other digit. A 9 is a loop with a descending tail, and that combination overlaps with several other digits at once, so it collides with a 4 when the loop fails to close, with a 7 when the loop is written small and tight, and with a 5 or an 8 depending on how the curve is drawn. The 4 and 9 pair taken in both directions is the single largest source of error at 10 of the 72. The 5 read as a 3 and the 7 read as a 2 follow the same pattern, where one curve or one crossbar is all that separates the two shapes.

The absence of augmentation also contributes to this. The model saw each training image in a single orientation, so it had no exposure to rotated, shifted or rescaled variants, and a digit written with an unusual slant falls outside the distribution it was fitted to.

Reading the misclassified grid directly shows that a number of these images are hard for a human to assign with confidence, with several written so that the intended digit is ambiguous from the shape alone. That places part of the remaining error on the handwriting rather than on the model.

7. Hyperparameter Discussion

The model has four configured values, a learning rate of 0.001, a batch size of 256, a dropout rate of 0.5 and a training length of ten epochs. No comparison across alternatives was run, so the effects described below are read from the single training run rather than measured against other settings.

The learning rate has the widest influence, since it sets how far each parameter moves per update. At 0.001 the loss fell from 2.3 to below 0.2 within the first 150 steps and then settled without the oscillation that signals too large a step. A larger rate risks overshooting and producing a loss curve that bounces, and a smaller one would descend more smoothly but need more than ten epochs to reach the same accuracy. Adam reduces the sensitivity of this choice, because it rescales each update by that parameter's own gradient history rather than applying one step size everywhere.

The batch size of 256 sets how many images contribute to each gradient estimate, which makes each epoch 235 updates and keeps the GPU working on large parallel operations, and training finished in about a minute as a result. It is also the direct cause of the noise band in both curves, since each plotted point summarizes 256 images rather than the full training set. A smaller batch would give noisier gradients and a longer run.

The dropout rate of 0.5 trades regularization strength against the capacity available on any single update, since the classifier sees a different random half of its 1,152 inputs at every step. A lower rate would let the layer use more of the representation and would likely raise training accuracy, and a higher rate would regularize harder with less of the representation available.

Ten epochs was enough without being wasteful. Accuracy passed 90 percent inside the first fifty steps and the band had stopped moving appreciably by around step 500, which is partway through the third epoch, so the remaining seven epochs refined rather than changed the model.

8. Conclusions & Suggested Improvements

8.1 Conclusions

The model reached 99.28 percent accuracy on the MNIST test set with 104,202 trainable parameters, three convolutional layers and one fully connected layer, trained in about a minute. That sits in the range normally reported for convolutional networks on this dataset. Dropout at 0.5 was the only regularization applied, and the comparison in section 6.1 shows a good fit with no gap in the direction overfitting would produce.

The clearest result from the analysis is where the remaining errors sit. The 72 misclassified images do not spread evenly across the ten digits but concentrate on 9, which accounts for 19 of them, and on pairs that share stroke geometry. The limit on accuracy is therefore ambiguity in the handwriting rather than a shortage of model capacity, which points improvement toward robustness in how a digit is drawn rather than toward more parameters.

The visualizations supported this. The second layer feature maps showed the digit separated into its component strokes across different channels, which is direct evidence that the network learned a hierarchy of features rather than memorizing training images.

8.2 Suggested Improvements

Data augmentation is the change that addresses the remaining errors most directly. Those errors concentrate on digits that share stroke geometry, with 9 alone accounting for 19 of the 72 and most often read as a 4 or a 5. Applying small random rotations, shifts and scale changes during training would expose the model to variation it never saw, which should help most on exactly those pairs, since a 9 misread as a 4 often differs from a cleanly written 9 only by its slant or by how tightly the loop closes. Augmentation also enlarges the effective training set at no labeling cost.

Batch normalization after each convolution and before its activation would improve efficiency rather than the error pattern. Normalizing activations per batch stabilizes the distribution entering each layer, which usually allows a higher learning rate and fewer epochs to reach the same accuracy.

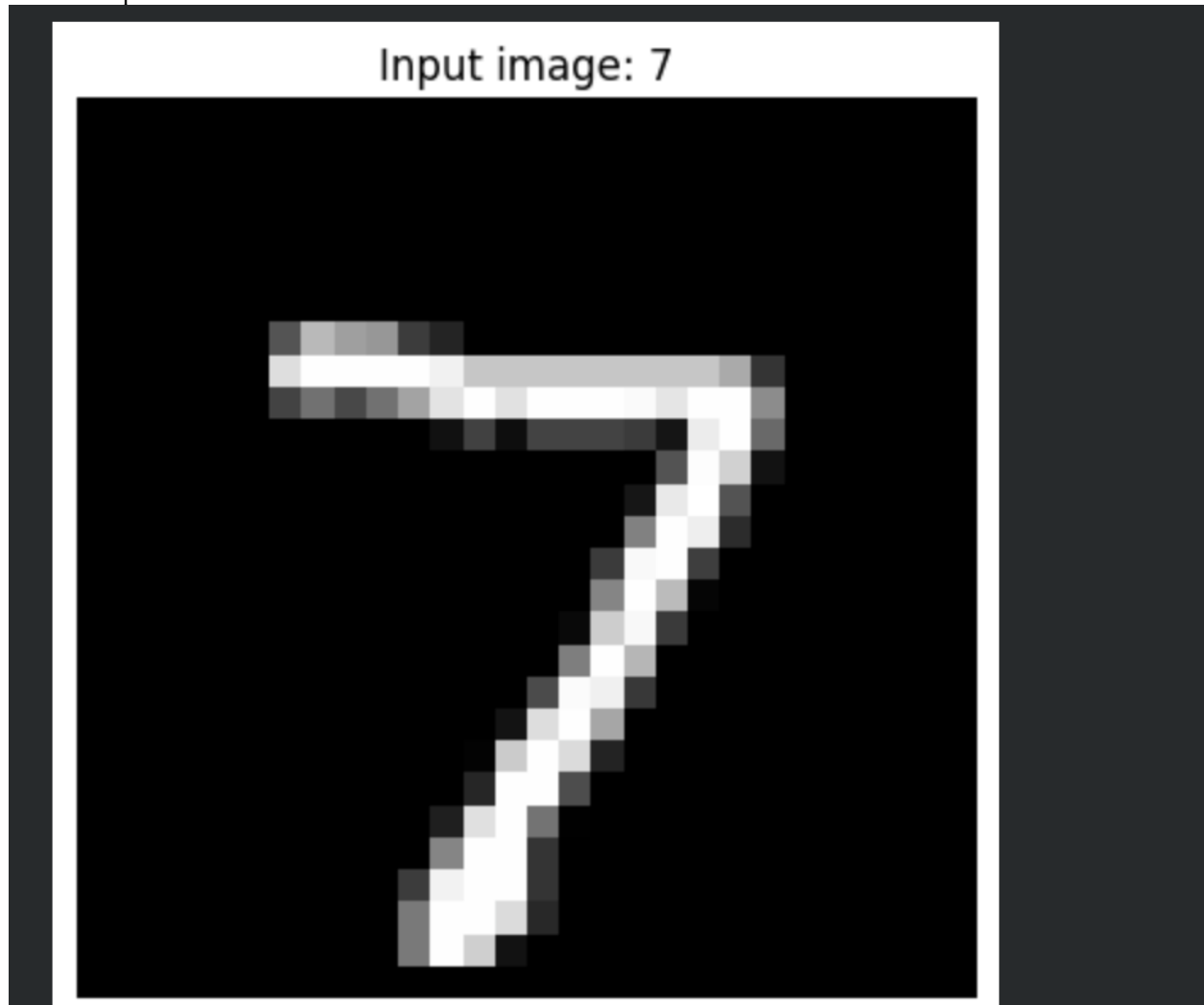
Two further changes address the limits of this report rather than of the model. A grid or random search over learning rate, batch size and dropout rate would replace values chosen by convention with values chosen by measurement, and averaging the predictions of several networks trained from different random initializations typically recovers a few tenths of a point, since independent models tend to fail on different borderline images.

9. Appendix — Source Code

[Link](#) to GitHub repository

- *AI transparency - I used Claude as an assistant on this project, both for debugging and refining the notebook code and for drafting and revising the written sections of this report. The architecture choices, the experimental results, the analysis and the conclusions are my own understanding of class material, and I reviewed and edited all generated code and text before including it.*

Feature map visuals:



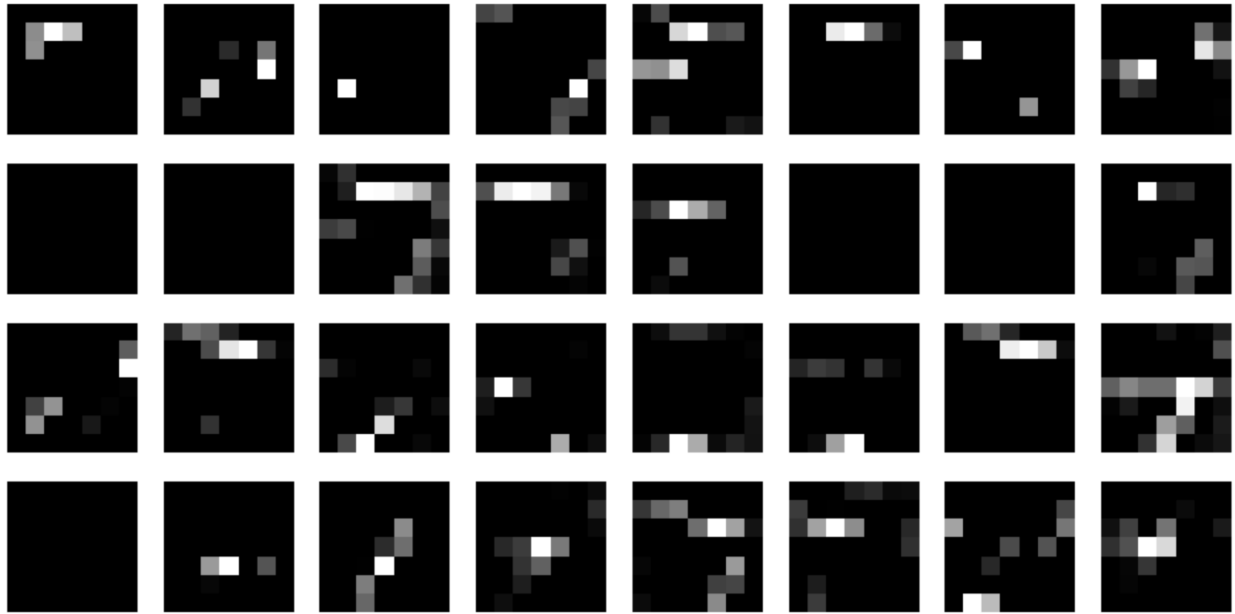
Conv layer 1 feature maps (first 32)



Conv layer 2 feature maps (first 32)



Conv layer 3 feature maps (first 32)



End of Report